

DOCUMENTAȚIE PROIECT POO

Simularea Traficului Rutier

Motor de simulare C++ • Server local Python (Flask) • Interfață web HTML/Canvas

Arhitectură pe trei niveluri: Engine → Server → Vizualizator

Limbaj: C++17 (MinGW g++ 15.2) | Python 3

Cuprins

1. Introducere	4
1.1. Scopul proiectului	4
1.2. Cerințele temei	4
1.3. Tehnologii utilizate.....	4
2. Arhitectura generală a sistemului.....	5
2.1. Cele trei niveluri.....	5
2.2. Fluxul de date.....	5
2.3. De ce este necesar serverul	5
3. Modelul orientat pe obiecte (C++).....	6
3.1. Ierarhia de clase	6
3.2. Clasa abstractă Vehicle	6
3.3. Polimorfismul în practică	6
3.3.1. Viteze maxime diferite	7
3.3.2. Comportament diferit la intersecții	7
3.3.3. Strategii de rutare diferite (edgeCost)	7
3.4. Compoziția: rețeaua rutieră.....	7
3.4.1. Topologia rețelei	7
4. Logica de simulare.....	9
4.1. Bucla principală (tick).....	9
4.2. Pathfinding — algoritmul lui Dijkstra.....	9
4.2.1. Problema rutării prin nodul central E	9
4.3. Semafoarele	9
4.4. Modelul de congestie (direcțional, bazat pe noduri).....	10
4.4.1. Regula.....	10
4.4.2. Exemplu de direcționalitate	10
4.4.3. Cazul a două vehicule care converg într-un nod liber	10
4.4.4. Ocupare „vie” (live occupancy).....	10
4.5. Coliziunile (accidente).....	10
4.5.1. Declanșarea.....	10
4.5.2. Efectul — îngheț și „tragere pe dreapta” (Opțiunea B)	11
4.6. Limitarea modelului — abstractizarea	11
5. Ruta de livrare multi-colet	12
6. Jurnalizare și comunicarea cu interfața	13

6.1. Clasa Logger	13
6.2. Structura unui snapshot.....	13
6.3. Serverul Flask	13
7. Interfața de vizualizare	14
7.1. Structura	14
7.2. Redarea pe Canvas.....	14
7.3. Elemente de claritate pentru utilizator.....	14
8. Setări și parametrizare	15
9. Testarea unitară	16
9.1. Teste de structură	16
9.2. Teste de mișcare	16
9.3. Teste de validare a rutelor	16
9.4. Compilarea și rularea testelor.....	16
10. Compilare, rulare și structura proiectului.....	17
10.1. Structura directoarelor	17
10.2. Compilarea motorului	17
10.3. Pornirea serverului.....	17
10.4. Erori frecvente întâmpinate și soluționate	17
11. Concluzii	18

1. Introducere

1.1. Scopul proiectului

Proiectul implementează o simulare a traficului rutier în care mai multe vehicule se deplasează pe o rețea de străzi respectând reguli de circulație simple. Vehiculele își calculează rutele în mod autonom, reacționează la semafoare, intră în congestie atunci când drumul din față este ocupat și pot fi implicate în accidente atunci când prea multe vehicule converg în același punct.

Spre deosebire de o simulare pur teoretică, sistemul este observabil în timp real: starea fiecărui tick este transmisă către o interfață web care redă animat deplasarea vehiculelor, starea semafoarelor, congestiile și accidentele.

1.2. Cerințele temei

Tema 3121A impune un set de cerințe obligatorii și un set de cerințe opționale. Tabelul de mai jos rezumă cerințele și gradul de îndeplinire în implementarea finală.

Cerință	Tip	Stadiu
Clase abstracte: Vehicle → Car, Motorcycle, Truck	Obligatorie	Îndeplinită
Polimorfism: viteze maxime și comportament la intersecții diferite	Obligatorie	Îndeplinită
Compoziție: Intersection, Street, RoadNetwork	Obligatorie	Îndeplinită
Evenimente jurnalizate: intrare/ieșire intersecție, coliziune	Obligatorie	Îndeplinită
Teste unitare pentru logica de mișcare și validarea rutelor	Obligatorie	Îndeplinită
Format cod + Git + jurnalizare	Obligatorie	Îndeplinită
Simulare în timp real cu pauză/accelerare	Opțională	Îndeplinită
Pattern Observer pentru notificarea evenimentelor	Opțională	Neimplementată

Tabel 1.1 — Maparea cerințelor temei pe implementarea finală.

1.3. Tehnologii utilizate

- C++17, compilat cu MinGW g++ 15.2.0, pentru motorul de simulare.
- Python 3 cu microframework-ul Flask, pentru serverul local care expune motorul către browser.
- HTML5, CSS și JavaScript (Canvas 2D) pentru interfața de vizualizare.
- Git pentru versionare; make / g++ pentru compilare.

2. Arhitectura generală a sistemului

2.1. Cele trei niveluri

Sistemul este structurat pe trei niveluri independente, fiecare cu o responsabilitate clară. Această separare a fost o decizie de proiectare esențială: motorul de simulare nu cunoaște nimic despre interfață, iar interfața nu conține logică de simulare.

Nivel	Tehnologie	Responsabilitate
Motor (Engine)	C++	Rulează simularea: pathfinding, deplasare, semafoare, congestie, coliziuni. Produce date structurate JSON.
Server (Bridge)	Python/Flask	Primește setările din browser, lansează executabilul C++ cu argumente, citește JSON-ul rezultat și îl returnează.
Vizualizator (Viewer)	HTML/JS/Canvas	Redă animat datele primite, tick cu tick. Nu simulează nimic.

Tabel 2.1 — Responsabilitățile celor trei niveluri.

2.2. Fluxul de date

La apăsarea butonului Start în interfață, are loc următoarea secvență:

1. Browserul trimite o cerere HTTP POST către server, la ruta `/run`, cu setările curente în corpul cererii (format JSON).
2. Serverul Flask construiește lista de argumente din linia de comandă și lansează `app.exe` cu aceste argumente.
3. Motorul C++ rulează întreaga simulare și scrie fișierul `log_day1.json`, conținând o fotografie (snapshot) a stării pentru fiecare tick.
4. Serverul citește fișierul JSON și îl returnează browserului ca răspuns la cererea POST.
5. Interfața redă datele tick cu tick, animând deplasarea vehiculelor pe Canvas.

Diagrama simplificată a fluxului: Browser → `/run` (POST) → Flask → `app.exe` → `log_day1.json` → Flask → Browser

2.3. De ce este necesar serverul

O întrebare firească este de ce nu se poate folosi un simplu server static (de exemplu Live Server din VS Code). Răspunsul ține de securitatea browserului: un browser nu poate lansa direct un executabil de pe disc, deoarece aceasta ar fi o vulnerabilitate majoră. Prin urmare este nevoie de un proces local — serverul Flask — care are dreptul să ruleze `app.exe` la cererea browserului și să îi întoarcă rezultatul. Un server static ar putea servi doar fișierul HTML, dar nu ar putea executa motorul C++.

3. Modelul orientat pe obiecte (C++)

3.1. Ierarhia de clase

Nucleul motorului este o ierarhie de clase care respectă principiile programării orientate pe obiecte cerute de temă: clase abstracte, polimorfism și compoziție.

Clasă	Rol	Relație
Vehicle	Clasă abstractă de bază pentru toate vehiculele	Bază a ierarhiei
Car	Vehicul care rutează pe distanța cea mai scurtă	Derivă din Vehicle
Motorcycle	Vehicul care rutează pe numărul minim de tronsoane	Derivă din Vehicle
Truck	Vehicul care evită străzile aglomerate	Derivă din Vehicle
Intersection	Nod al rețelei, cu semafor	Compoziție în RoadNetwork
Street	Tronson între două intersecții	Compoziție în RoadNetwork
RoadNetwork	Graful complet al rețelei rutiere	Conține intersecții și străzi
Simulation	Orchestratorul buclei de simulare	Deține rețeaua, vehiculele, logger-ul
Logger	Jurnalizarea evenimentelor și export JSON	Folosit de Simulation

Tabel 3.1 — Clasele principale ale motorului.

3.2. Clasa abstractă Vehicle

Clasa Vehicle este abstractă: conține metode pur virtuale care obligă fiecare subclasă să își definească propriul comportament. Astfel, codul de simulare poate trata uniform orice vehicul prin pointer la clasa de bază, fără a-i cunoaște tipul concret — esența polimorfismului.

Metodele pur virtuale definite în Vehicle sunt:

```
virtual int  getMaxSpeed() = 0;
virtual void behaviorAtIntersection(Intersection* i) = 0;
virtual int  edgeCost(Street* s) = 0;
virtual int  getTickRate() = 0;
```

Pe lângă acestea, clasa de bază oferă logica comună tuturor vehiculelor: deplasarea efectivă între noduri (move), verificarea sosirii la destinație (hasArrived), precum și câmpurile partajate pentru așteptare în trafic (waitTicks), pentru evitarea blocajelor (lastWaitedStreet) și pentru îngheț în urma unui accident (freezeTicks).

3.3. Polimorfismul în practică

Cele trei tipuri de vehicule diferă prin trei aspecte concrete, toate exprimate prin redefinirea metodelor virtuale.

3.3.1. Viteze maxime diferite

Vehicul	getMaxSpeed()	Observație
Car / Delivery	120 km/h	Viteză de referință
Motorcycle	160 km/h	Cel mai rapid
Truck	90 km/h	Cel mai lent

Tabel 3.2 — Vitezele maxime polimorfice. Vitezele influențează ritmul de animație în interfață.

3.3.2. Comportament diferit la intersecții

- Car: se oprește complet la roșu (`currentSpeed = 0`), circulă la viteză maximă la verde.
- Motorcycle: la roșu încetinește la jumătate din viteza maximă (`getMaxSpeed() / 2`), nu se oprește complet.
- Truck: se oprește complet la roșu, prudent, dar circulă normal la verde.

Observație de implementare: o versiune anterioară a Motorcycle folosea `currentSpeed / 2`, ceea ce dădea zero la primul tick (vehiculul pornea cu viteza 0) și îngheța vehiculul. Corecția a fost folosirea `getMaxSpeed() / 2`.

3.3.3. Strategii de rutare diferite (edgeCost)

Costul unui tronson în algoritmul de drum minim depinde de tipul vehiculului, prin metoda `edgeCost`. Aceasta determină ce înseamnă „drumul optim” pentru fiecare:

Vehicul	edgeCost(Street)	Strategie de rutare
Car / Delivery	lungimea tronsonului	Drumul cel mai scurt ca distanță
Motorcycle	1 (constant)	Numărul minim de tronsoane (fewest hops)
Truck	nivelul de congestie	Evită străzile aglomerate

Tabel 3.3 — Funcția de cost determină strategia de rutare per tip de vehicul.

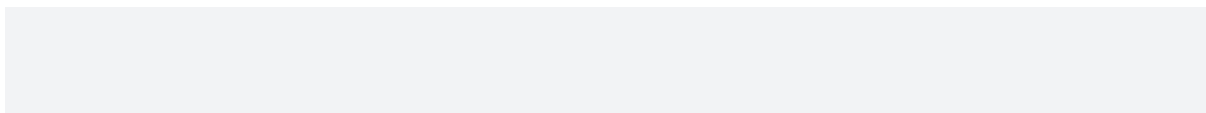
3.4. Compoziția: rețeaua rutieră

Rețeaua este modelată prin relații de compoziție clare (relații „are-un”), folosind pointeri inteligenți `std::shared_ptr` pentru gestionarea automată a memoriei:

- `RoadNetwork` conține nodurile (`Intersection`) și tronsoanele (`Street`).
- Fiecare `Intersection` deține lista tronsoanelor conectate la ea, necesară pentru pathfinding.
- Fiecare `Street` cunoaște cele două capete (`endA`, `endB`) și lista vehiculelor aflate pe ea.

3.4.1. Topologia rețelei

Rețeaua este o grilă 3×3 cu 9 intersecții notate de la A la I și 12 tronsoane bidirecționale. Dispunerea nodurilor este următoarea:



A	---	B	---	C
D	---	E	---	F
G	---	H	---	I

Tronsoanele orizontale (de exemplu AB, BC) au lungimea 100 și limita de viteză 50 km/h, iar cele verticale (de exemplu AD, DG) au lungimea 120 și limita 60 km/h. Nodul central E este singurul cu patru conexiuni (DE, EF, BE, EH), ceea ce îl face punctul prin care trec majoritatea rutelor optime — un detaliu important, care a generat un set de probleme tratate într-o secțiune ulterioară.

Cele patru colțuri ale grilei — A, C, G și I — sunt singurele puncte de plecare și destinație ale vehiculelor.

4. Logica de simulare

4.1. Bucla principală (tick)

Simularea avansează în pași discreți numiți tick-uri. La fiecare tick, clasa `Simulation` parcurge toate vehiculele și, pentru fiecare, decide ce se întâmplă în ordinea de priorități de mai jos:

6. Dacă vehiculul este înghețat în urma unui accident (`freezeTicks > 0`), decrementează contorul și rămâne pe loc, marcat „CRASHED”.
7. Dacă vehiculul de fundal a ajuns la destinație, primește o nouă destinație aleatoare (roaming) ca să continue să circule.
8. Dacă vehiculul așteaptă în trafic (`waitTicks > 0`), decrementează contorul și rămâne pe loc, marcat „TRAFFIC”.
9. Aplică comportamentul la intersecție (semafor): dacă viteza devine 0, se oprește la roșu („RED”).
10. Calculează următorul tronson de pe rută; verifică regula de congestie; dacă poate, se deplasează.

La finalul fiecărui tick, după ce toate vehiculele s-au deplasat: semafoarele comută, se detectează coliziunile, se construiește snapshot-ul JSON și se verifică livrarea coletelor.

4.2. Pathfinding — algoritmul lui Dijkstra

Rutarea este realizată cu algoritmul lui Dijkstra în `RoadNetwork::findPath`, folosind o coadă de priorități. Costul muchiilor este dat polimorfic de `edgeCost`, deci același algoritm produce rute diferite pentru fiecare tip de vehicul.

4.2.1. Problema rutării prin nodul central E

Versiunea inițială a algoritmului nu marca nodurile finalizate (nu avea o mulțime `visited`). Pe un nod cu patru muchii, cum este E, acest defect permitea ca nodul să fie extras de mai multe ori din coada de priorități cu costuri învechite, alegând uneori un predecesor greșit. Rezultatul: rute rupte sau care evitau incorect centrul, iar vehiculele se blocau la E.

Corecția a constat în adăugarea unei mulțimi `visited` care marchează fiecare nod ca finalizat o singură dată, ignorând intrările învechite din coadă, plus oprirea curată a reconstrucției drumului la nodul de start:

```
if (visited.count(current)) continue; // sare peste intrările învechite
visited.insert(current);
// ...
if (next == nullptr) continue;
if (visited.count(next)) continue; // nod deja finalizat
```

Corectitudinea drumului minim este verificată ulterior prin teste unitare (vezi capitolul 7).

4.3. Semafoarele

Fiecare intersecție are un semafor cu o stare binară (roșu / verde). Durata fiecărei faze este aleatoare, între 1 și 3 tick-uri, și este re-aleasă la fiecare schimbare de fază. Detalii importante de implementare:

- Comutarea semafoarelor se face la SFÂRȘITUL tick-ului, după ce vehiculele s-au deplasat. Astfel un vehicul vede întotdeauna faza completă a semaforului în timpul deciziei sale.
- Starea inițială (roșu sau verde) este aleatoare la fiecare rulare, dacă setarea „Random Start Lights” este activă; altfel toate pornesc pe verde.
- Contorul de tick-uri rămase este transmis în interfață și afișat sub fiecare nod (de exemplu „2t”).

4.4. Modelul de congestie (direcțional, bazat pe noduri)

Modelul de congestie a evoluat semnificativ. Versiunea finală — denumită intern „Opțiunea C” — este realistă și ușor de înțeles: un vehicul așteaptă doar dacă nodul către care se îndreaptă este deja ocupat.

4.4.1. Regula

- Nodul țintă este liber → vehiculul se deplasează, fără întârziere.
- Nodul țintă are deja 1 vehicul → vehiculul așteaptă 1 tick (marcat „TRAFFIC”).
- Nodul țintă are 2 sau mai multe vehicule → vehiculul așteaptă 2 tick-uri.

Regula este direcțională: așteaptă doar vehiculul care se apropie de un nod ocupat. Vehiculul aflat deja în acel nod, sau cel care se îndepărtează de el, nu este afectat.

4.4.2. Exemplu de direcționalitate

Considerăm un vehicul aflat în A care merge spre D și un al doilea vehicul aflat în B care merge spre A. Al doilea vehicul se îndreaptă spre A, care este ocupat (primul vehicul stă acolo), deci al doilea așteaptă. Primul vehicul se îndepărtează spre D, deci nu suferă nicio întârziere. Astfel, doar al doilea vehicul intră în trafic — comportament intuitiv și corect.

4.4.3. Cazul a două vehicule care converg într-un nod liber

Dacă două vehicule sosesc simultan într-un nod gol (de exemplu unul din A și unul din G spre D, cu D liber), niciunul nu așteaptă — ambele intră. Acest lucru evită necesitatea unei reguli arbitrare de „cine are prioritate”. Traficul apare doar dacă nodul era deja ocupat înainte de sosire.

4.4.4. Ocupare „vie” (live occupancy)

O rafinare ulterioară a făcut ca harta de ocupare a nodurilor să fie actualizată pe măsură ce vehiculele se deplasează în cadrul aceluiași tick, nu fixată la începutul tick-ului. Astfel, în momentul în care vehiculul din față eliberează un nod, acel nod devine imediat liber pentru vehiculul din spate, iar o coloană de trafic înaintează firesc. (Un șir de vehicule „bară la bară” avansează totuși câte un vehicul pe tick, asemenea unei unde reale de trafic care se propagă spre spate.)

4.5. Coliziunile (accidente)

Inițial, o coliziune era doar înregistrată în jurnal, fără efect. În versiunea finală, coliziunile au consecințe reale și sunt unificate conceptual cu modelul de trafic.

4.5.1. Declanșarea

O coliziune se produce atunci când 3 sau mai multe vehicule sfârșesc tick-ul în ACELAȘI nod. Regula este fixă și clară: convergența a trei vehicule în același punct = accident.

4.5.2. Efectul — îngheț și „tragere pe dreapta” (Opțiunea B)

Vehiculele implicate într-un accident:

- Sunt înghețate pentru un număr aleator de 2–3 tick-uri (freezeTicks), marcate „ CRASHED”.
- Sunt „trase pe dreapta”: scoase din tronsonul curent, deci nu mai contează ca ocupând spațiu. Restul traficului trece nestingherit pe lângă ele, evitând efectul de bulgăre de zăpadă (snowball).
- Își păstrează destinația: la expirarea înghețului, reiau deplasarea de unde au rămas.

Această abordare (denumită intern „Opțiunea B”) a fost aleasă în detrimentul uneia în care vehiculele înghețate ar fi continuat să blocheze tronsonul — variantă care putea provoca blocaje în lanț de zeci de tick-uri.

4.6. Limitarea modelului — abstractizarea

Este important de menționat onest că simularea este o abstractizare, nu o simulare fizică. Tronsoanele nu au benzi sau sens de circulație propriu-zis; un „tronson” este pur și simplu un segment între două noduri. Prin urmare, regulile de congestie și coliziune nu modelează fizica reală a unei coliziuni, ci suprapunerea abstractă a vehiculelor în noduri. Această decizie a fost asumată conștient, pentru a păstra modelul simplu și clar.

5. Ruta de livrare multi-colet

Vehiculul special DELIVERY (cyan) nu efectuează o singură cursă, ci o rută completă de livrare formată din mai multe colete.

- La începutul rulării primește un număr aleator de colete (între 3 și 5).
- Pornește dintr-un colț aleator și se îndreaptă spre prima destinație.
- La sosire, „livrează” coletul, apoi alege un nou colț ca următoare destinație, și tot așa.
- Rularea se încheie la livrarea ultimului colet, sau la atingerea numărului maxim de tick-uri.

Pentru ca destinația să poată fi schimbată în timpul rulării, clasa Vehicle a primit metoda `setDestination`. Deoarece `hasArrived` compară poziția curentă cu destinația, schimbarea destinației face automat ca vehiculul să nu mai fie „sosit”, declanșând recalcularea rutei spre noul colț.

Interfața afișează progresul livrării: un contor „Pachete X/Y” în bara de statistici, descrierea în limbaj natural în banner-ul de stare și o linie în jurnal la fiecare livrare („ Pachet 2/4 livrat la G la tick-ul 11”).

Vehiculele de fundal (Car, Motorcycle, Truck) circulă continuu (roaming): la atingerea destinației, își aleg imediat un nou colț, menținând harta animată pe toată durata rulării.

6. Jurnalizare și comunicarea cu interfața

6.1. Clasa Logger

Logger-ul scrie două fluxuri în paralel:

- Un fișier text lizibil (log_dayN.txt) cu evenimentele cerute de temă: intrare în intersecție, ieșire din intersecție, coliziune, livrare.
- Un fișier structurat JSON (log_dayN.json) cu câte un snapshot complet al stării pentru fiecare tick — sursa de date pentru interfață.

6.2. Structura unui snapshot

Fiecare snapshot (TickSnapshot) conține tot ce are nevoie interfața pentru a reda un cadru:

- Numărul tick-ului.
- Pentru fiecare vehicul: identificator, poziție, destinație, stare (moving / RED / TRAFFIC / CRASHED / done) și viteză.
- Pentru fiecare intersecție: identificator, dacă este pe roșu, câte tick-uri mai rămân din fază.
- Lista coliziunilor din tick-ul respectiv.
- Starea livrării: câte colete au fost livrate și câte sunt în total.

6.3. Serverul Flask

Serverul expune câteva rute. Cele esențiale sunt:

Rută	Metodă	Rol
/	GET	Servește interfața (ui/index.html)
/run	POST	Compilează la nevoie, rulează app.exe cu setările, întoarce JSON-ul ambelor zile
/status	GET	Raportează dacă executabilul este compilat

Tabel 6.1 — Rutele serverului local.

La ruta /run, serverul citește setările din corpul cererii și le transformă în argumente de linie de comandă pentru executabil, în ordinea: maxTicks, congThresh, deliveryImmune, deliveryImmuneCol, randomLights.

7. Interfața de vizualizare

7.1. Structura

Interfața este o aplicație web cu o singură pagină, organizată în patru file (tab-uri): Simulation, Info, History și Settings.

- Simulation: bara de unelte (Start / Pause / Reset, slider de viteză), bara de statistici (Tick, Pachete, Coliziuni, Opriri la roșu, Întârzieri trafic), canvas-ul mare cu harta, panoul vehiculelor și jurnalul de evenimente.
- Info: explică tipurile de vehicule și regulile, cu o legendă vizuală.
- History: istoricul rulărilor anterioare.
- Settings: setările care se trimit motorului C++ la următoarea pornire.

7.2. Redarea pe Canvas

Interfața nu simulează; ea redă datele primite. La încărcare, citește numărul total de tick-uri din date pentru a afișa o bară de progres onestă. Fiecare vehicul este desenat ca o formă distinctă (mașină, motocicletă, camion), interpolat („glisat”) fluid între noduri. Ritmul de glisare este proporțional cu viteza vehiculului (km/h) primită din C++, astfel încât motocicleta se deplasează vizibil mai repede decât camionul.

S-a aplicat și o corecție pentru ecranele de înaltă rezoluție (high-DPI): canvas-ul este randat la rezoluția fizică a ecranului și scalat prin contextul de desen, eliminând aspectul pixelat.

7.3. Elemente de claritate pentru utilizator

- Un banner de stare în limbaj natural, deasupra hărții, descrie ce face vehiculul de livrare („Delivery conduce de la B spre G (pachet 3/4”).
- O legendă permanentă sub hartă explică culorile vehiculelor și semnificația semafoarelor.
- Ținta curentă de livrare este marcată vizibil: inele pulsatorii, halou luminos, un pin 📍 care plutește și eticheta „DELIVER HERE”.
- Insignele de pe vehicule indică starea: RED, TRAFFIC sau 🚨 CRASHED (controlabile prin setarea „Show Badges”).

8. Setări și parametrizare

Setările din interfață sunt transmise prin lanțul complet HTML → Flask → C++, astfel încât modifică efectiv comportamentul simulării (nu doar afișarea). Modificările se aplică la următoarea apăsare a butonului Start, întrucât simularea este pre-calculată integral înainte de redare.

Setare	Efect asupra simulării
Max Ticks	Plafonul de tick-uri al buclei C++. Dacă livrarea nu se termină, rularea se oprește (DNF).
Show Badges on Map	Pur vizual: afișează/ascunde etichetele RED / TRAFFIC / CRASHED pe hartă.
Delivery Immune to Congestion	Când e activă, vehiculul de livrare ignoră întârzierile de trafic.
Delivery Immune to Collisions	Când e activă, vehiculul de livrare nu îngheață niciodată într-un accident.
Random Start Lights	Când e activă, semafoarele pornesc aleator; altfel, toate pe verde.

Tabel 8.1 — Setările și efectul lor real asupra motorului.

Cele două imunități ale vehiculului de livrare sunt independente: se poate configura un vehicul care evită traficul dar poate fi implicat în accidente, sau invers. Verificarea funcționării lanțului se poate face urmărind linia „Config: ...” afișată de C++ în fereastra serverului la fiecare pornire.

9. Testarea unitară

Testele sunt grupate în trei categorii și acoperă explicit logica de mișcare și validarea rutelor, conform cerinței obligatorii. Toate cele 13 teste trec.

9.1. Teste de structură

- `getOtherEnd`: un tronson întoarce corect capătul opus față de cel dat.
- Semafor: starea inițială este verde, iar comutarea funcționează corect.
- Construcția rețelei: există 9 intersecții și 12 tronsoane; nodurile inexistente întorc `nullptr`.

9.2. Teste de mișcare

- Un vehicul pornește la nodul de start și nu este „sosit”.
- După deplasarea pe un tronson, vehiculul ajunge la nodul corect.
- Vehiculul este marcat „sosit” când ajunge la destinație.
- `setDestination` resetează corect starea de sosire (esențial pentru livrarea multi-colet).
- Vitezele polimorfice sunt corecte (120 / 160 / 90).

9.3. Teste de validare a rutelor

- Există o rută între colțuri opuse ($A \rightarrow I$).
- Ruta minimă $A \rightarrow I$ are exact 4 tronsoane (confirmă că Dijkstra găsește drumul cel mai scurt).
- Start = destinație întoarce o rută goală (caz limită tratat).
- Noduri adiacente ($A \rightarrow B$) întorc exact un tronson, care chiar le conectează.
- Parcurgerea completă a rutei calculate aduce efectiv vehiculul la destinație (validează mișcarea și rutarea împreună).

9.4. Compilarea și rularea testelor

```
g++ -std=c++17 -Wall -o tests src/tests.cpp src/Street.cpp \
    src/Intersection.cpp src/RoadNetwork.cpp src/Vehicle.cpp \
    src/Car.cpp src/Motorcycle.cpp src/Truck.cpp \
    src/Simulation.cpp src/Logger.cpp
./tests
```

10. Compilare, rulare și structura proiectului

10.1. Structura directoarelor

```
F:\P00\
app.exe           (executabilul compilat)
server.py         (serverul Flask)
Makefile, README.md
log_day1.json / .txt (ieșirile simulării)
src\              (toate fișierele .cpp / .h)
ui\index.html     (interfața web)
```

10.2. Compilarea motorului

```
cd F:\P00
g++ -std=c++17 -Wall -o app src/main.cpp src/Street.cpp \
    src/Intersection.cpp src/RoadNetwork.cpp src/Vehicle.cpp \
    src/Car.cpp src/Motorcycle.cpp src/Truck.cpp \
    src/Simulation.cpp src/Logger.cpp
```

10.3. Pornirea serverului

```
cd F:\P00
& "E:\Thonny\Thonny\python.exe" server.py
# apoi se deschide http://localhost:5000 în browser
```

Dependențele Python (Flask și Flask-CORS) se instalează o singură dată cu:

```
& "E:\Thonny\Thonny\python.exe" -m pip install flask flask-cors
```

10.4. Erori frecvente întâmpinate și soluționate

- „multiple definition of RoadNetwork::build” — apărea când conținutul unui fișier era lipit accidental în altul; soluția a fost re-copiarea fișierului corect.
- „undefined reference to Intersection::...” — apărea când Intersection.cpp conținea din greșeală directiva `#pragma once` la început, fiind tratat ca antet și necompilat; soluția a fost eliminarea directivei.
- pip lipsă pe Python 3.14 — interpretorul de sistem era defect; soluția a fost folosirea interpretorului Python livrat cu Thonny.

11. Concluzii

Proiectul a evoluat de la două componente separate și neconectate (o simulare în JavaScript și un proiect C++) la un sistem unitar, coerent, pe trei niveluri, în care motorul C++ este creierul real al simulării, iar interfața web este un vizualizator fidel al rezultatelor sale.

Toate cerințele obligatorii ale temei sunt îndeplinite: ierarhia de clase abstracte, polimorfismul prin viteze, comportament la intersecție și strategii de rutare diferite, compoziția rețelei rutiere, jurnalizarea evenimentelor și testele unitare pentru mișcare și validarea rutelor. Dintre cerințele opționale, simularea în timp real cu pauză și accelerare este implementată.

Dincolo de cerințe, sistemul include funcționalități suplimentare: rută de livrare multi-colet, trafic de fundal continuu, model de congestie direcțional emergent, accidente cu efect de îngheț, un lanț complet de parametrizare a setărilor și o interfață web clară, prietenoasă cu utilizatorul începător.

Singura cerință neimplementată este pattern-ul Observer, care este opțional. O direcție viitoare de dezvoltare ar fi refactorizarea jurnalizării într-un mecanism subiect–observator, alături de extinderea grilei sau introducerea benzilor de circulație direcționale pentru a transforma coliziunile dintr-o abstractizare într-un model fizic propriu-zis.